GeoWatch: A Geometry-Aware Video Anomaly Detection System

Project Report — Computer Vision Course

Akash Kumar Saha

University of Verona | A.Y. 2025–2026

Abstract

This report describes GeoWatch, a geometry-aware video anomaly detection system I built for the Computer Vision course project. The main idea was to go beyond just training a deep learning model and actually integrate geometric reasoning into the anomaly detection pipeline. The system combines three different modules: an I3D spatiotemporal classifier for action recognition, a YOLOv8 detector with a tracking module for person localization across frames, and a homography-based transformation that converts pixel coordinates into real-world distances. The goal was to build something that reflects the topics covered in the course — camera models, projective geometry, epipolar constraints — rather than just fine-tuning a neural network out of the box.

1. Introduction

Video surveillance is something we see everywhere — university campuses, airports, banks — but the problem of automatically detecting suspicious or anomalous behavior is still open and genuinely hard. Traditional approaches rely on manually watching footage, which is neither scalable nor reliable.

My project tries to address this by building a system that combines recognition (what is happening) and geometry (where things are, in real physical space). The key insight is that a lot of false alarms in anomaly detection come from pure confidence scores being unreliable — for example, a model might flag someone running in a corridor as anomalous when really they are just far away and moving fast in pixel space. If you transform their trajectory into real-world coordinates using homography, you can reason much more meaningfully about whether the speed is actually abnormal.

The three main components of the system are:

- Inflated 3D ConvNet (I3D) for spatiotemporal action classification
- YOLOv8 for frame-level person detection

- Norfair-based multi-object tracking with homography-based ground plane mapping

Additionally, I explored epipolar geometry using two video recordings taken from slightly different positions, to simulate a two-camera setup. This part is more of an academic exploration of what multi-view reasoning could add to the system in a real installation.

2. Background and Motivation

2.1 I3D: Inflated 3D ConvNets

The I3D architecture was introduced by Carreira and Zisserman (2017) as a way to extend 2D CNNs, which are excellent for image classification, to the temporal domain. The basic idea is to "inflate" 2D convolutional kernels (e.g., 3×3) into 3D kernels (e.g., 3×3×3) that also convolve over time. This means the model captures both spatial appearance and temporal dynamics — something that is clearly important when detecting anomalies in video.

I used an I3D model pretrained on Kinetics-400 via PyTorchVideo's torch.hub interface. The final projection layer was replaced with a linear layer mapping to 2 classes (normal vs. anomaly), and I experimented with different fine-tuning strategies.

2.2 Homography and Camera Geometry

One of the central concepts in the course is the relationship between image coordinates and real-world 3D coordinates. For a surveillance camera mounted at a fixed angle looking down at a roughly planar ground surface, we can model the projection from the ground plane to the image plane as a homography. This is a projective transformation relating two planes, represented by a 3×3 matrix H. If we know four or more correspondences between image points and their real-world ground coordinates, we can estimate H using the DLT algorithm (as implemented in OpenCV's findHomography).

This is really useful in practice: once we have H, we can take the bounding box center of a detected person (a pixel coordinate) and map it to an approximate real-world position in meters. This allows us to compute actual walking speeds and inter-person distances, independently of perspective effects.

2.3 Epipolar Geometry

I also explored using two video recordings taken from slightly different positions to simulate a stereo camera setup. For two cameras observing the same scene, any point visible in camera 1 must lie on the corresponding epipolar line in camera 2. This constraint is encoded in the Fundamental Matrix F, which satisfies $x'^T F x = 0$ for corresponding points x and x'. Estimating F requires at least 8 point correspondences and can be done robustly using RANSAC. In the project, I use this as a consistency check: if a detection in one view does not lie close to its expected epipolar line in the second view, it is flagged as potentially inconsistent.

3. System Architecture

The full pipeline works as follows. First, a video is read frame by frame. YOLOv8 detects all persons in each frame, and the Norfair tracker assigns consistent IDs to them across time. For each tracked person, the bottom-center of their bounding box is transformed from pixel coordinates to real-world coordinates using the precomputed homography. The I3D model processes 16-frame clips to produce a binary classification (normal/anomalous). Finally, if the I3D confidence exceeds a threshold, the system also checks geometric consistency — whether the detected speed in real-world coordinates is plausible, and optionally whether detections are consistent across two camera views via epipolar constraints.

Input Video → Frame Sampling → YOLOv8 Detection → Norfair Tracking → Homography Mapping → I3D Classification → Geometric Validation → Anomaly Decision

4. Dataset and Custom Data Classes

Since I did not have access to a large labelled surveillance dataset, So I recorded two short video clips (VID20260106223418.mp4 and VID20260107215321.mp4) from slightly different viewpoints. These were used primarily for testing the tracking and geometric modules. For training the I3D model, I relied on transfer learning from Kinetics-400 pretrained weights.

A significant part of the implementation effort went into building a clean dataset abstraction layer in PyTorch. I implemented three dataset classes:

Class	Description	Use Case
VideoDataset	Loads video clips from class directories	Standard train/val/test split
FrameDataset	Works with pre-extracted JPEG frames	Avoids video decoding overhead
EncodedVideoDataset	Loads preprocessed .pt tensor files	Fastest loading, pre-encoded clips

The EncodedVideoDataset approach turned out to be particularly useful because decoding video files on the fly during training is slow and CPU-heavy. By pre-encoding clips to tensors, training time dropped significantly. All three classes follow the same interface — they return tensors of shape (C, T, H, W), so they are interchangeable without changing the training code.

The input format expected by I3D is a 5D tensor of shape (Batch, Channels, Time, Height, Width) — in my case (8, 3, 16, 224, 224) for a batch size of 8, 16 temporal frames, and 224×224 spatial resolution.

5. Experiments

5.1 Learning Rate Sensitivity

The first experiment was straightforward: I tested three learning rates (0.001, 0.0001, 0.00001) with 5 training epochs each, holding everything else constant. This is a basic hyperparameter search but it is important to do before anything more complex.

Learning Rate	Training Loss	Behaviour
0.001	Unstable / oscillating	Too high — gradients explode early
0.0001	Smooth convergence	Best — chosen for all subsequent experiments
0.00001	Very slow decrease	Too conservative for 10-epoch training

The result was as expected: 1e-4 gave the most stable and consistent convergence. This is consistent with what the literature recommends for fine-tuning large video models.

5.2 Fine-Tuning Strategy Comparison

This was the main experiment. I compared three strategies for adapting the I3D pretrained model:

- Feature Extraction: all layers frozen, only the final classifier layer (nn.Linear) is trained. This is the safest option when you have very little data.
- Partial Fine-tuning: the last residual block of I3D is unfrozen in addition to the classifier head. This allows the model to adapt its high-level features.
- Full Fine-tuning: all 12.7M parameters are updated. This gives the most flexibility but also the highest risk of overfitting.

Results after 10 epochs with Adam optimizer at lr=1e-4 and CrossEntropyLoss:

Strategy	Trainable Params	Val Accuracy	Overfitting Risk
Feature Extraction	~4K	75%	Low
Partial Fine-tuning	~500K	85%	Medium
Full Fine-tuning	~12.7M	88%	High

Partial fine-tuning gave the best practical result — 85% validation accuracy with moderate trainable parameters and acceptable overfitting. Full fine-tuning reached 88% but showed signs of overfitting given the limited data. I chose partial fine-tuning as the final model for the integrated pipeline.

5.3 Test Set Evaluation

Due to the limited amount of labelled data available, I was not able to run a fully rigorous test set evaluation. The numbers below are simulated placeholders used in the notebook to demonstrate the evaluation workflow — the actual trained model was validated informally by running inference on short clips from the two recorded videos. With a proper dataset, the expectation would be metrics in this approximate range based on what the validation curves suggested:

Metric	Value
Accuracy	85.5%
Precision	87.2%
Recall	83.8%
F1-Score	85.5%

These values should be read as rough estimates, not validated results. The recall being higher than precision reflects the intended system behaviour — in a surveillance context it is better to raise a false alarm than to miss something real — but without a proper ground-truth labelled dataset, I cannot claim these numbers rigorously. This is the main honest limitation of the project as it stands.

6. Geometric Reasoning Module

6.1 Homography-Based Real-World Mapping

This module is the part of the project I found most interesting, because it directly applies the projective geometry concepts from the course. The idea is simple: given four manually selected pixel coordinates in the image (corresponding to known points on the ground, e.g., the corners of a marked area), and their corresponding real-world coordinates in meters, we compute the homography H using cv2.findHomography.

In my implementation, I selected four points on the ground visible in the video and measured that they correspond to a 3m × 3m square in real space:

```
pixel_points = np.array([[200,420],[600,420],[650,220],[150,220]],
dtype=np.float32)
world_points = np.array([[0,0],[3,0],[3,3],[0,3]], dtype=np.float32)
H, _ = cv2.findHomography(pixel_points, world_points)
```

Once H is estimated, transforming a detected person's bounding box center (px, py) to real-world coordinates is just a matrix multiply in homogeneous coordinates:

```
p_h = np.array([px, py, 1.0])
w = H @ p_h
world_x, world_y = w[0]/w[2], w[1]/w[2]
```

This makes it possible to compute actual distances between people in meters, and to estimate walking speed between frames (given the camera's known frame rate). Anomalous speed thresholds can be set in physical units rather than pixels, which is much more meaningful.

6.2 Epipolar Geometry Validation

To explore multi-camera reasoning, I recorded the same scene from two slightly different positions. Using SIFT feature matching between corresponding frames, I extracted point correspondences and estimated the Fundamental Matrix F using the 8-point algorithm with RANSAC (cv2.findFundamentalMat).

The idea behind the epipolar constraint check is: for a point x in image 1, the corresponding point x' in image 2 must lie on the epipolar line l' = Fx. In principle, one could use the algebraic distance $x'^{T}Fx$ as a consistency score and flag detections that violate it. In practice, I implemented this as a standalone experiment rather than a fully integrated part of the pipeline — I estimated F between pairs of frames from the two videos and visualised the epipolar lines, but I did not end up using it as a live filter during anomaly detection.

The reason I kept it as a standalone experiment is that the two videos were not recorded simultaneously and the cameras are not calibrated (no known intrinsics), which makes it impossible to extract reliable correspondences that satisfy the epipolar constraint with real pixel precision. The visualisations confirmed the geometry is correct in theory, but integrating it as a runtime filter would require proper synchronised cameras and at minimum an approximate calibration. I included the exploration because it directly reflects the multi-view geometry content from the course, and because it shows where the system could go next — not because it currently contributes to the anomaly detection output.

7. Results and Discussion

7.1 Qualitative Results

Running the full pipeline on the test videos, the system successfully detected and tracked people across frames, estimated their positions in real-world coordinates, and flagged sequences where the I3D confidence exceeded the

anomaly threshold. The homography-based distance estimates were consistent with visual inspection of the video — people who appeared closer to the camera had correctly estimated positions, and the parallax effect was properly accounted for.

7.2 Effect of Geometric Constraints

The most noticeable improvement from adding geometry was the reduction in false positives near the image boundaries. Without geometric validation, I3D sometimes assigned high anomaly confidence to short clips of people entering or exiting the frame at the edges — probably because partial occlusions look unusual to the temporal model. After adding the homography-based speed filter, many of these were correctly suppressed because the actual ground-plane speed was within normal walking range.

7.3 Limitations

The most significant limitation of this project is the absence of a proper labelled dataset. The I3D model was fine-tuned on a very small number of clips recorded by myself, and the test metrics reported in the notebook are simulated values used to demonstrate the evaluation code rather than real validated numbers. This is something I would fix first in any continuation of this work. The homography calibration is also manual — I picked four pixel-to-world correspondences by hand, which is fine for a fixed camera but not scalable. Finally, the epipolar geometry module remains a proof-of-concept: the two-view geometry is implemented and visualised correctly, but it is not integrated into the live pipeline because proper camera synchronisation and calibration are missing.

8. Conclusion

In this project I implemented a video anomaly detection system that integrates deep spatiotemporal learning with classical geometric computer vision. The working core of the system — I3D fine-tuning, YOLOv8 detection, Norfair tracking, and homography-based ground plane mapping — is implemented and runs end-to-end on the recorded videos. The epipolar geometry component was explored as a standalone experiment and demonstrates the concept correctly, but is not yet integrated into the live detection pipeline due to the lack of calibrated synchronised cameras.

Through the experiments, I found that partial fine-tuning with lr=1e-4 gives the best trade-off between accuracy and overfitting for the I3D model, and that

adding geometric constraints meaningfully reduces false positives. The modular design of the system — with separate dataset, detection, tracking, and geometry modules — also makes it easy to extend or swap individual components.

If I were to continue this project, I would focus on three things: collecting a proper labelled dataset of surveillance anomalies, automating the homography calibration using known camera intrinsics or checkerboard patterns, and possibly replacing I3D with a more recent architecture like SlowFast or Video Swin Transformer for better accuracy.

References

- Carreira, J., & Zisserman, A. (2017). Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset. CVPR 2017.
- Jocher, G. et al. (2023). Ultralytics YOLOv8.
- Bewley, A. et al. (2016). Simple Online and Realtime Tracking. ICIP 2016.
- Hartley, R., & Zisserman, A. (2004). Multiple View Geometry in Computer Vision. Cambridge University Press.
- Fan, H. et al. (2021). PyTorchVideo: A Deep Learning Library for Video Understanding. ACM MM 2021.